



Peer to Peer Networks

By Cathy Chen
CMSC 621, Fall 2007

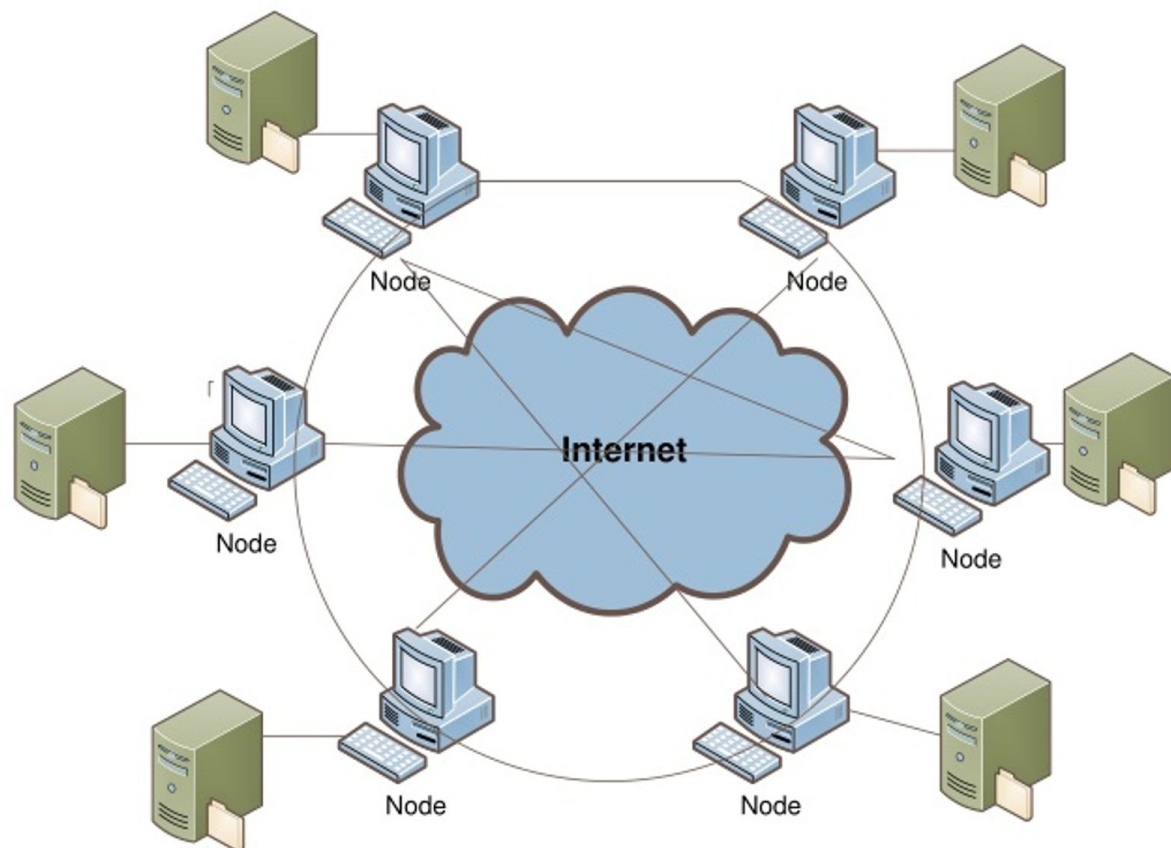
Outline

- What is P2P Network?
- Overlay Network centralization
 - Purely Decentralized Architectures
 - Partially Centralized Architectures
 - Hybrid Decentralized Architectures
- Network Structure
 - Unstructured
 - Structured
- Unstructured Architectures
 - Hybrid Decentralized
 - Purely Decentralized
 - Partially Centralized
- Structured Architectures
 - Freenet - A Loosely Structured System
 - Chord
 - Can (Content Addressable Network)
 - Tapestry

What is P2P Network?

- A distributed network composed of a large number of distributed, heterogeneous, autonomous, and highly dynamic peers in which participants share a part of their own resources such as processing power, storage capacity, software, and file contents.
- The participants in the P2P network can act as a server and a client at the same time.
- They are accessible by other nodes directly, without passing intermediary entities.

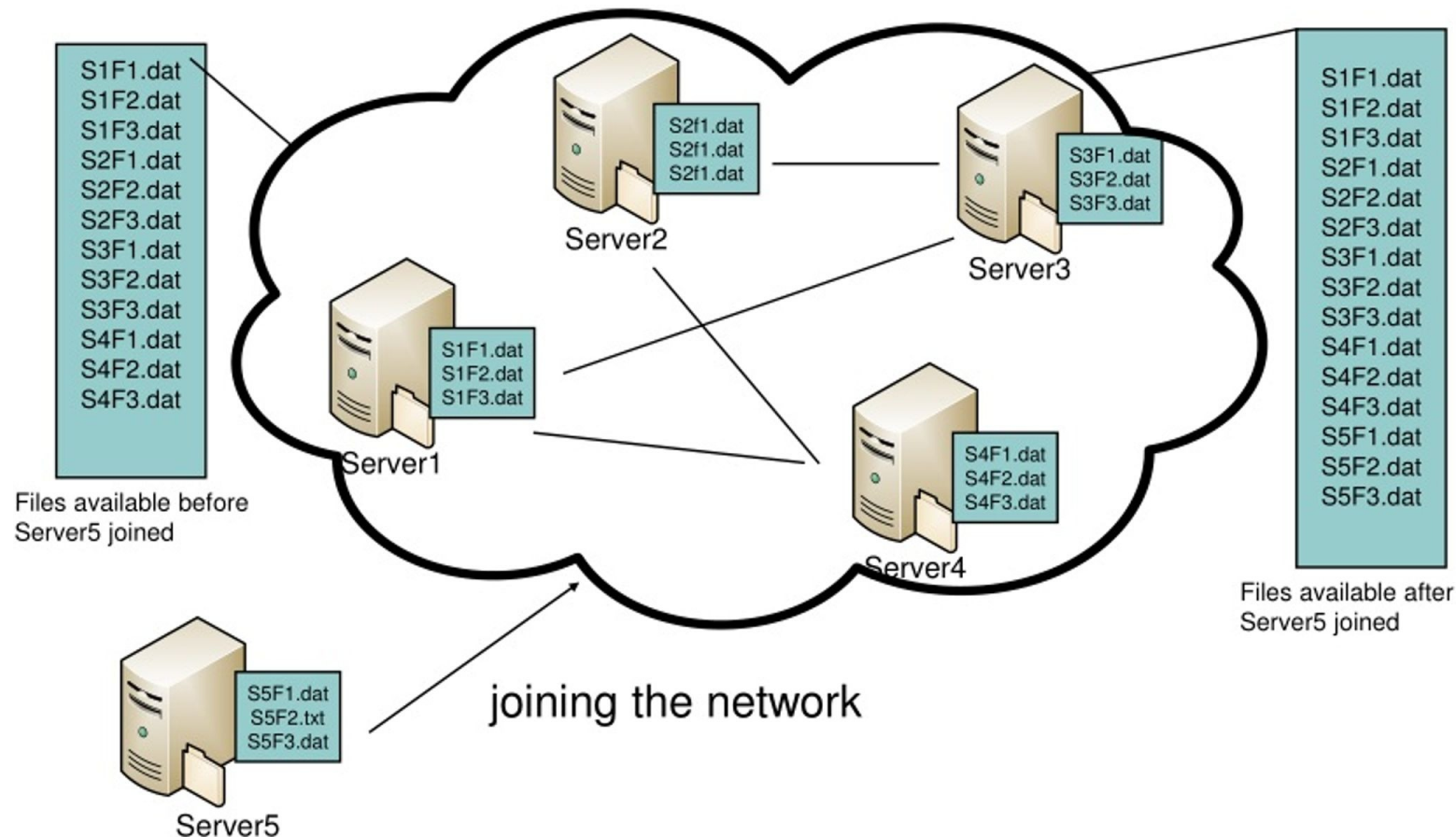
P2P Network



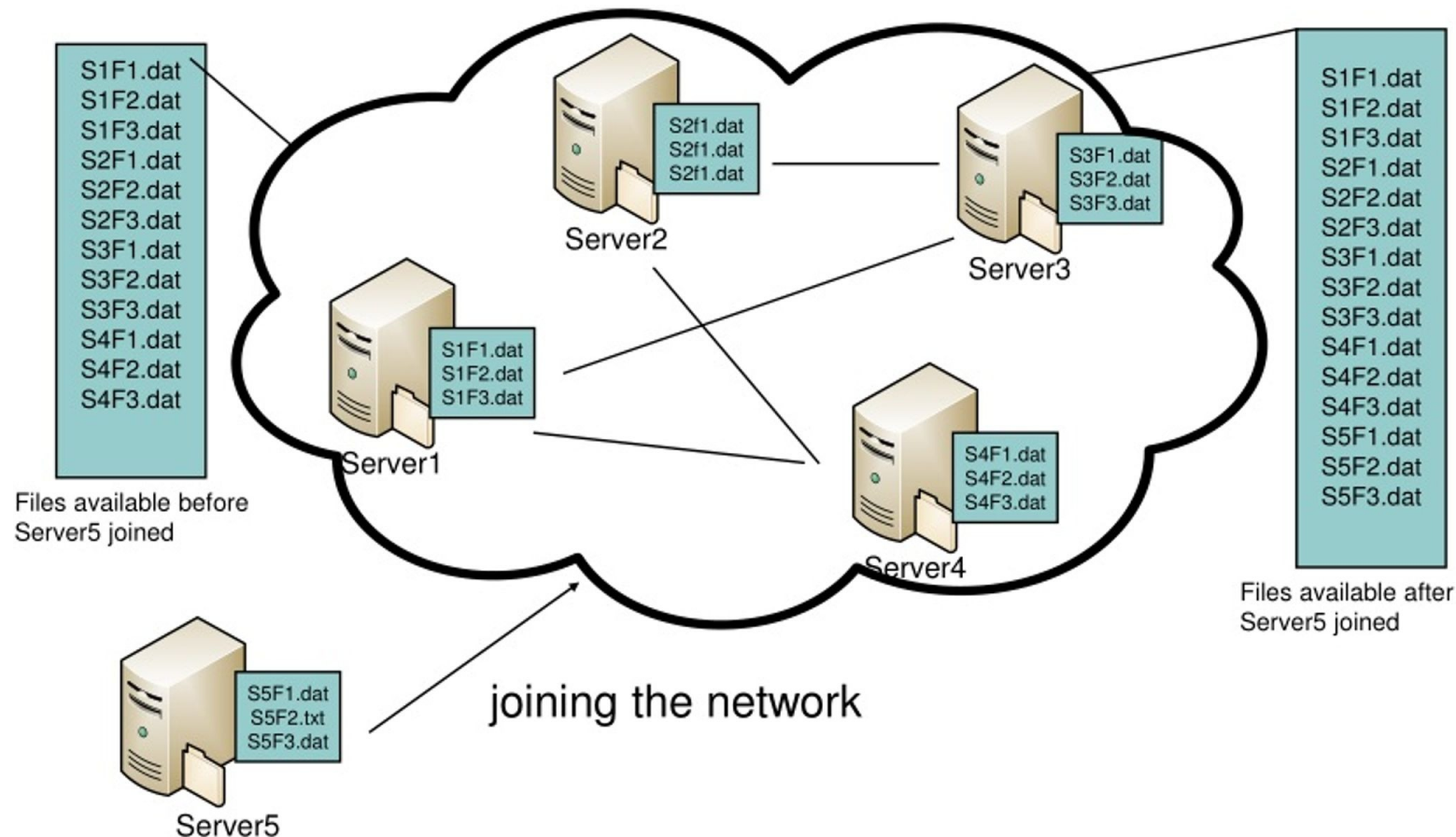
Peer-to-peer Architectures

- Although in purest form peer-to-peer overlay networks are supposed to be totally decentralized, in practice this is not always true, and systems with various degrees of centralization are encountered.
- They are identified into the following three categories:
 - Purely Decentralized architectures
 - Hybrid decentralized architectures
 - Partially Centralized architectures

Purely Decentralized Architectures



Purely Decentralized Architectures



Gnutella : Pure Decentralized System

- peer-to-peer networking: applications connect to peer applications
- focus: decentralized method of searching for files
- each application instance serves to:
 - store selected files
 - route queries (file searches) from and to its neighboring peers
 - respond to queries (serve file) if file stored locally
- How it works:
 - Searching by flooding:
 - If you don't have the file you want, query 7 of your partners.
 - If they don't have it, they contact 7 of their partners, for a maximum hop count of 10.
 - Requests are flooded, but there is no tree structure.
 - No looping but packets may be received twice.

Advantages and Disadvantages of Purely Decentralized Systems

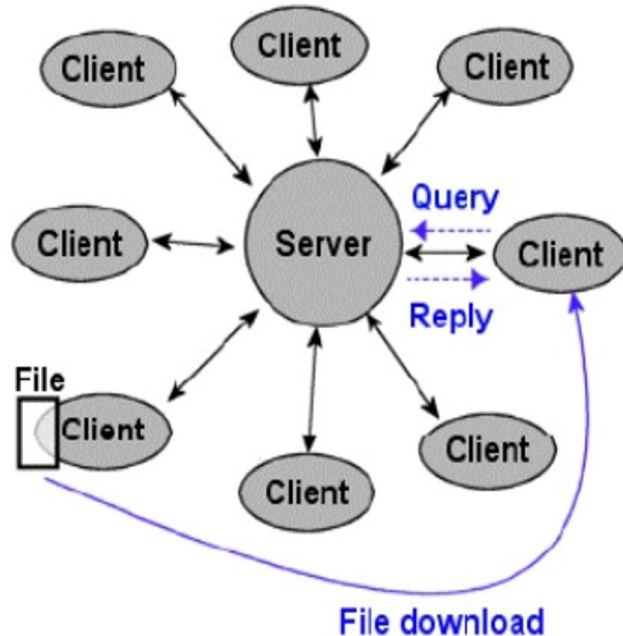
Advantages

- Inherently fault-tolerant, since there is no central point of failure and the loss of a peer or even a number of peers can easily be compensated.
- Have a greater degree of autonomous control over their data and resources

Disadvantages

- Slow discovery time
- No guarantee about quality of services

Hybrid Decentralized Architectures



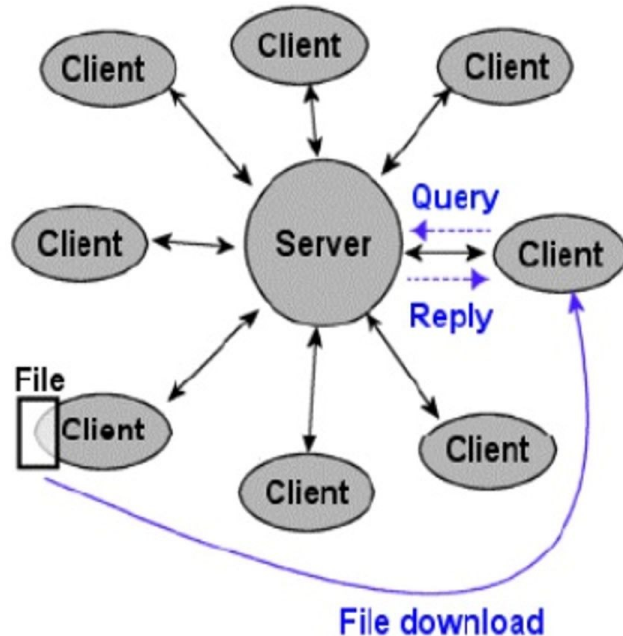
- Client computers send requests for files to the server.

- The server searches for matches in its index, returning a list of users that hold the matching file.

- The user then opens direct connections with one or more of the peers that hold the requested file, and downloads it

- Ex. Napster

Hybrid Decentralized Architectures



- Client computers send requests for files to the server.

- The server searches for matches in its index, returning a list of users that hold the matching file.

- The user then opens direct connections with one or more of the peers that hold the requested file, and downloads it

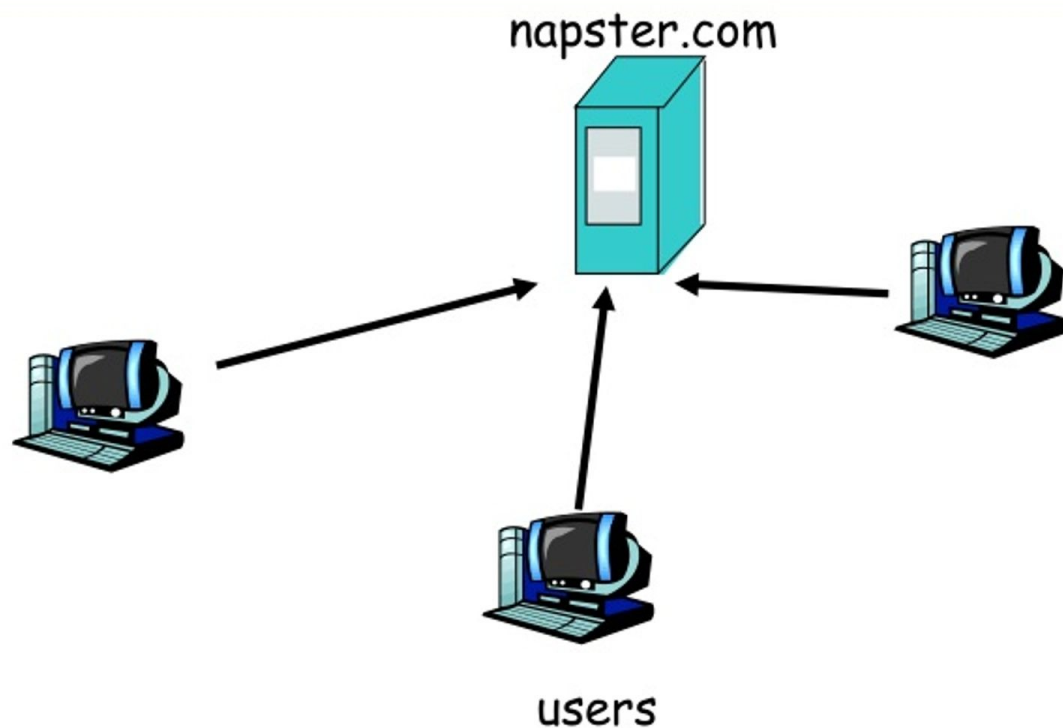
- Ex. Napster

Napster

- Application-level, client-server protocol over point to-point TCP, centralized system
- Retrieval: four steps
 - Connect to Napster server
 - Upload your list of files (push) to server.
 - Give server keywords to search the full list with.
 - Select “best” of correct answers. (pings)
- centralized server: single logical point of failure, can load balance among servers using DNS rotation, potential for congestion
- no security: passwords in plain text, no authentication, no anonymity

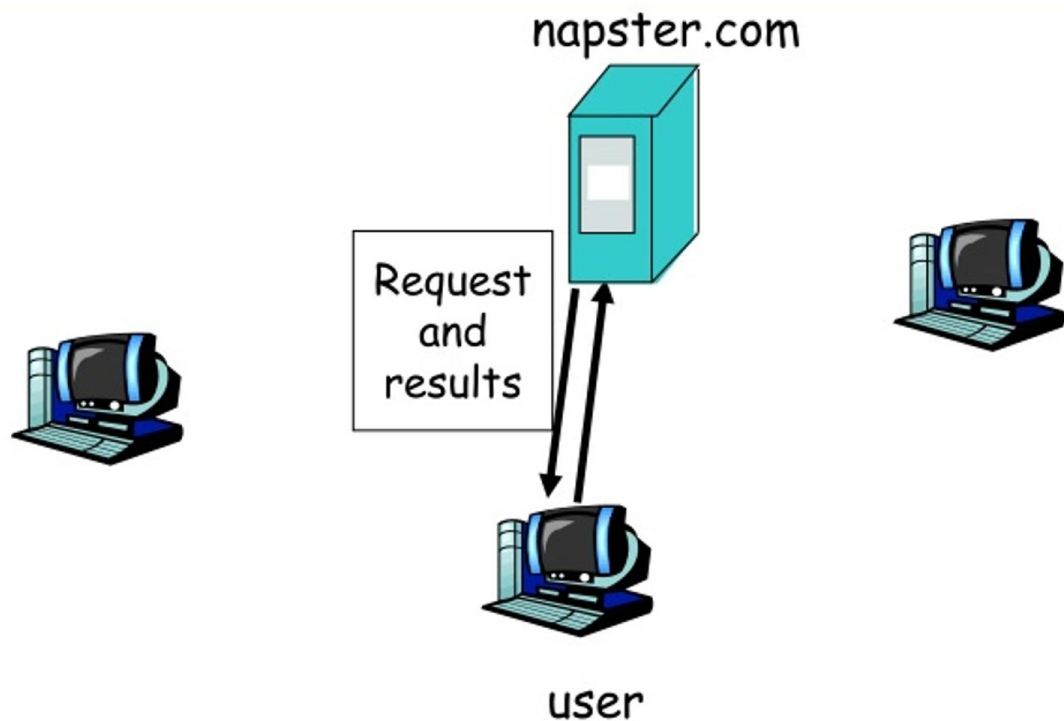
Napster: How it works?(1)

1. File list is uploaded



Napster: How it works?(2)

2. User requests search at server.

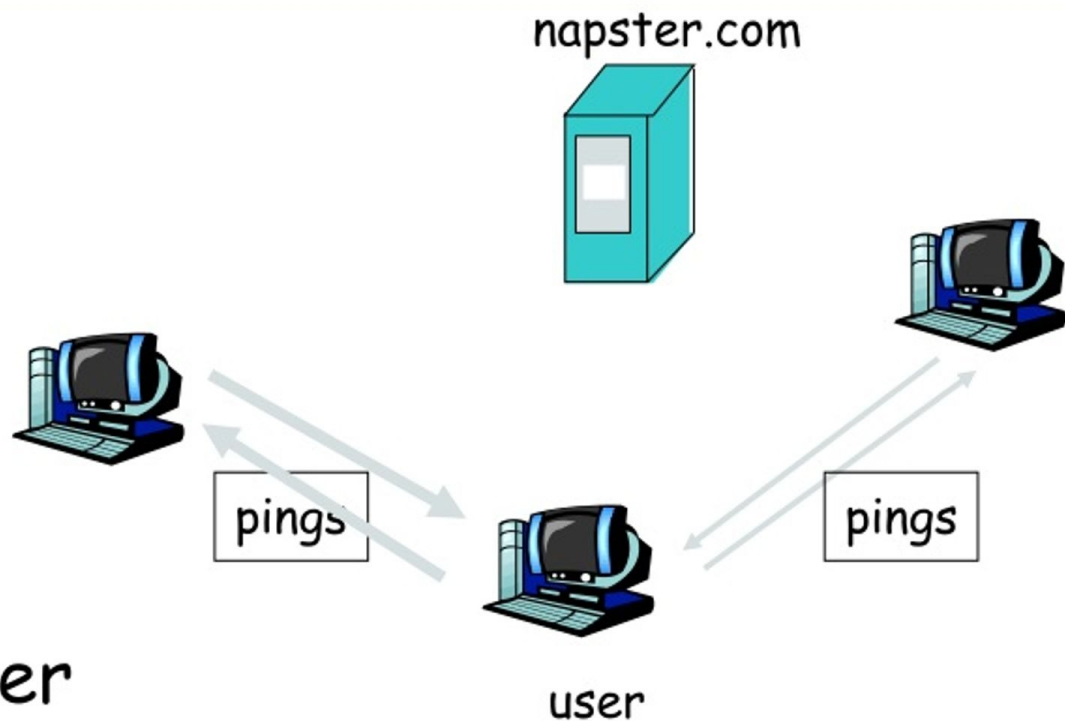


Napster: How it works?(3)

3.

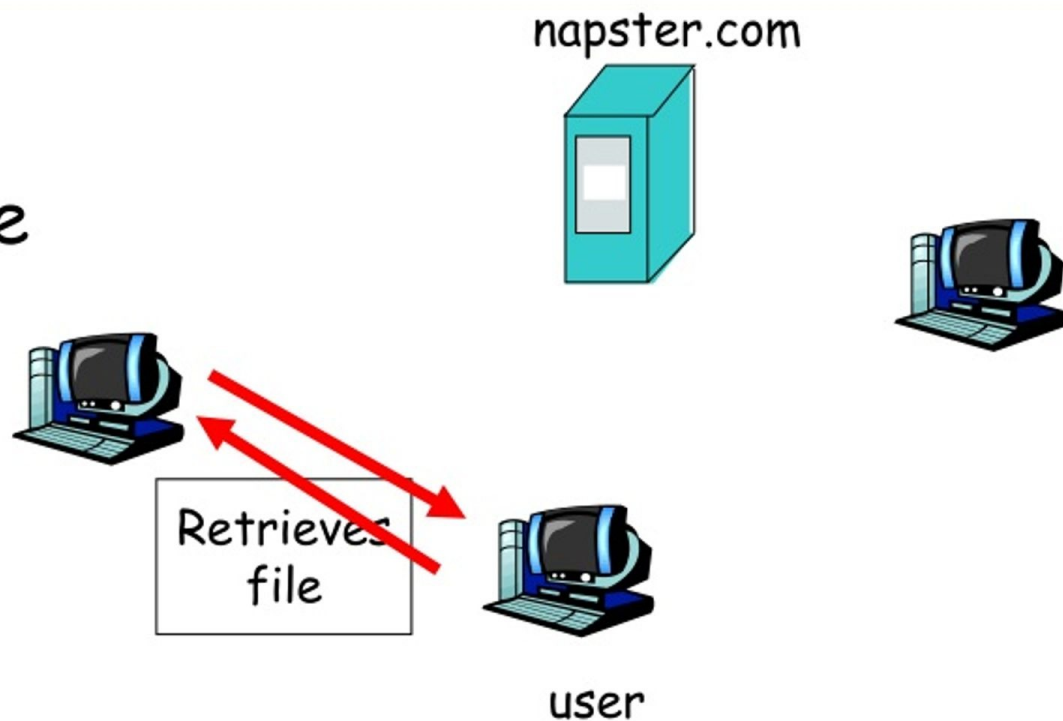
User pings
hosts that
apparently
have data.

Looks for
best transfer
rate.



Napster: How it works?(4)

4. User retrieves file



Advantages and Disadvantages of Hybrid Decentralized Systems

Advantages

- Easy to implement
- Locate files quickly and efficiently

Disadvantages

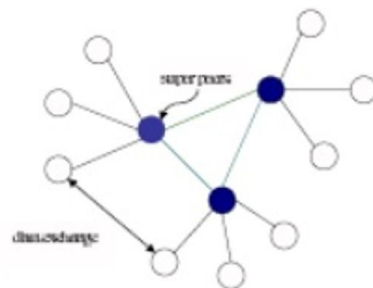
- Vulnerable to censorship, legal action, surveillance, malicious attack, and technical failure
- Considered inherently unscalable as they are bound to be limitations to the size of the server database and its capacity to respond to queries.

Partially Centralized Architectures

- use the concept of supernodes: nodes that are dynamically assigned the task of servicing a small subpart of the peer network by indexing and caching files contained therein.
- Peers are automatically elected to become supernodes if they have sufficient bandwidth and processing power.
- Supernodes index the files shared by peers connected to them, and proxy search requests on behalf of these peers.
- All queries are therefore initially directed to supernodes.

Partially Centralized Systems

- Kazaa and Morpheus are two similar partially centralized systems which use the concept of “SuperNodes”.
- In Morpheus a central server provides new peers with a list of one or more Supernodes with which they can connect. SuperNodes index the files shared by peers connected to them, and proxy search requests on behalf of these peers. Queries are therefore sent to SuperNodes, not to other peers.
- Recent version of the Gnutella protocol.
 - A mechanism for dynamically selecting supernodes organizes the Gnutella network into an interconnection of *superpeers* (as they are referred to) and client nodes. When a node with enough CPU power joins the network, it immediately becomes a *superpeer* and establishes connections with other superpeers, forming a flat unstructured network of superpeers. If it establishes a minimum required number of connections to client nodes within a specified time, it remains a *superpeer*. Otherwise it turns into a regular client node.



Advantages of Partially Decentralized Systems

Advantages

- In comparison with purely decentralized systems, discovery time and the traffic on messages exchanging between nodes are reduced
- In comparison with Hybrid Decentralized Systems, workload on central server are reduced, but with slower information discovery
- The inherent heterogeneity of peer-to-peer networks is taken advantage of and exploited
- no unique point of failure
 - If one or more supernodes go down, the nodes connected to them can open new connections with other supernodes, and the network will continue to operate.
 - Peers can become supernodes themselves even when all supernodes go down

Discovery Mechanisms for P2P systems

- Distributed peer-to-peer systems often require a discovery mechanism to locate specific data within the system.
- P2P systems have evolved from first generation centralized structures to second generation flooding-based and then third generation systems based on distributed hash tables:
 - Centralized indexes and repositories
 - Flooding broadcast of queries
 - Routing Model

Centralized Indexes and Repositories

- Used in hybrid system
- peers of the community connect to a centralized directory servers, which store all information regarding location and usage of resources
- A central directory server maintains:
 - an index with meta data (file name, time of creation etc.) of all files in the network
 - a table of registered user connection information (IP addresses, connection speeds etc.)
 - a table listing the files that each user holds and shares in the network

Flooding broadcast of queries

- Simple and robust
 - No state maintenance needed
 - High tolerance to node failures
- Effective and of low latency
 - Always find the shortest/fastest routing paths
- Fundamental Operation for
 - Broadcasting in distributed systems
 - P2P communications

Problems of Flooding

- Loops in Gnutella networks
 - Caused by redundant links
 - Result in endless message routing
- Current solution by Gnutella
 - Detect and discard redundant messages
 - Limit TTL(time-to-live) of messages
- Unnecessary traffic is still too much
 - The redundant links are still there

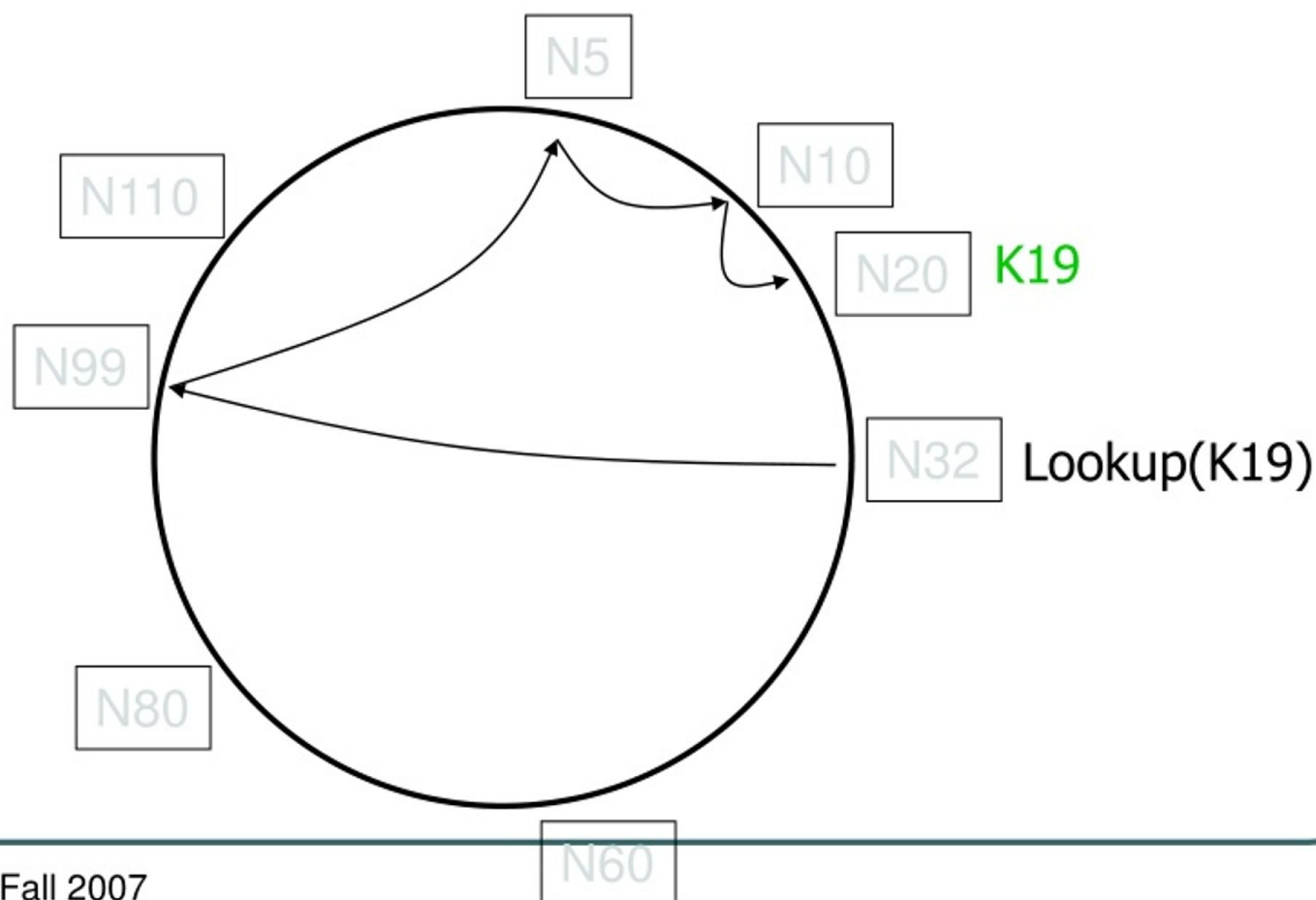
Routing Model

- ◉ Adds structure to the way information about resources are stored using distributed hash table
- ◉ Provides a mapping between the resource identifier and location
- ◉ Reduces the number of p2p hops that must be taken to locate a resource

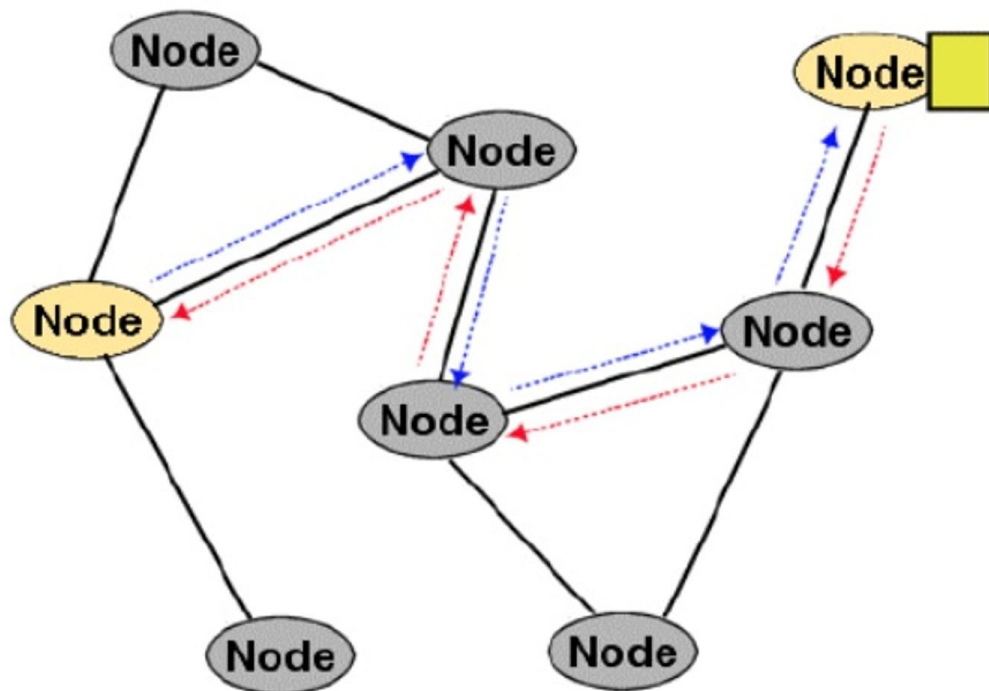
Chord

- Provides peer-to-peer hash lookup service:
 - Lookup(key) $\rightarrow$ IP address
 - Chord does not store the data
- Efficient: $O(\log N)$ messages per lookup
 - N is the total number of servers
- Scalable: $O(\log N)$ state per node
- Robust: survives massive changes in membership

Chord: Lookup Mechanism



Freenet



- The query is forwarded from node to node using the routing table, until it reaches the node which has the requested data.
- The replay is passed back to the original node following the reverse path.

Pastry

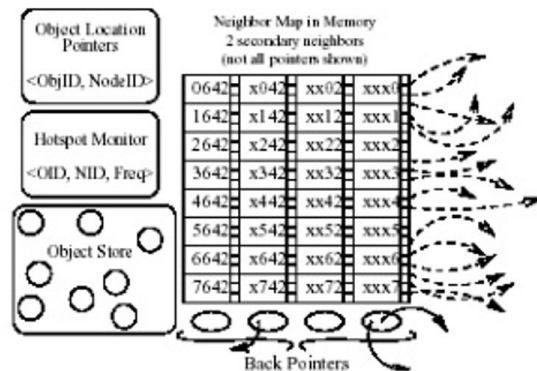
- ◉ Completely decentralized, scalable, and self-organizing; it automatically adapts to the arrival, departure and failure of nodes.
- ◉ Seeks to minimize the distance messages travel, according to a scalar proximity metric like the number of IP routing hops.
- ◉ In a Pastry network,
 - Each node has a unique id, nodeid.
 - Presented with a message & a key, Pastry node efficiently routes the message to the node with a nodeid that is numerically closest to the key.

CAN (Content Addressable Network)

- The network is created in a tree-like form.
- Each node is associated to one in the upper level and to a group in the lower level.
- A query travels from the uppermost level down through the network until a match is found or until it reaches the lowermost level.
- For its query model, scalability is an issue.

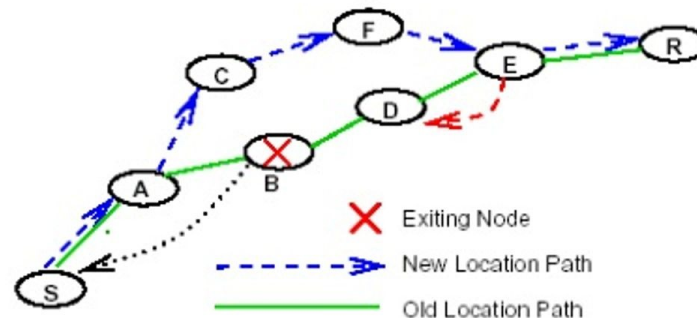
Tapestry

- Self-administered, self-organized, location independent, scalable, fault-tolerant
- Each node has a neighbor map table with neighbor information.



Tapestry Cont.

- The system is able to adapt to network changes because its algorithms are dynamic.
- This also provides for Fault-handling



When to consider unstructured P2P systems

- The placement of content (files) is completely unrelated to the overlay topology.
- In an unstructured network, content typically needs to be located. Searching mechanisms range from brute force methods such as flooding the network with propagating queries in a breadth-first or depth-first manner until the desired content is located, to more sophisticated and resource-preserving strategies that include the use of random walks and routing indices. The searching mechanisms employed in unstructured networks have obvious implications, particularly in regards to matters of availability, scalability, and persistence.
- Unstructured systems are generally more appropriate for accommodating highly transient node populations.
- Some representative examples of unstructured systems are Napster, Publius, Gnutella, Kazaa and Freehaven, et.

When to consider Structured P2P systems

- In structured networks the overlay topology is tightly controlled and files (or pointers to them) are placed at precisely specified locations. These systems essentially provide a mapping between content (e.g. file identifier) and location (e.g. node address), in the form of a distributed routing table, so that queries can be efficiently routed to the node with the desired content.
- Structured systems offer a scalable solution for exact-match queries, i.e. queries in which the exact identifier of the requested data object is known (as compared to keyword queries). Using exact-match queries as a substrate for keyword queries remains an open research problem for distributed environments
- A disadvantage of structured systems is that it is hard to maintain the structure required for efficiently routing messages in the face of a very transient node population, in which nodes are joining and leaving at a high rate.
- Typical examples of structured systems include Chord, CAN, Tapestry et cetera.

Conclusion

This table shows a classification of peer-to-peer content distribution systems and location and routing infrastructures in terms of their Network structure, with some typical examples.

	Centralization		
	Hybrid	Partial	None
Unstructured	Napster, Publius	Kazaa, Morpheus, Gnutella, Edutella	Gnutella, FreeHaven
Structured Infrastructures			Chord, CAN, Tapestry, Pastry
Structured Systems			OceanStore, Mnemosyne, Scan, PAST, Kademlia, Tarzan

References

- B. Pourebrahimi, K. bertels and S. Vassiliadis. A survey of Peer-toPeer Networks, http://ce.et.tudelft.nl/publicationfiles/1075_526_prorisc05.pdf
- Ion Stoica, Robert morris, david Karger, M. Frans Kaashoek and Hari Balakrishnan. Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications, http://pdos.csail.mit.edu/papers/chord:sigcomm01/chord_sigcomm.pdf
- Ismail Guvenc and Juan Jose Urdaneta. Peer to Peer File Sharing: A Survey, www.cs.princeton.edu/courses/archive/fall02/cs597C/P2P/p2psurvey.ppt
- Nelson Mina. Distributed Systems Topologies: Part 1, http://www.openp2p.com/pub/a/p2p/2001/12/14/topologies_one.html
- Peer-to-Peer, http://en.wikipedia.org/wiki/Peer-to-peer#Applications_of_peer-to-peer_networks
- Stephanos Androutsellis-Theotokis and Diomidis Spinellis. A survey of peer-to-peer content distribution technologies. ACM Computing Surveys, 36(4):335–371, December 2004.

Questions?

